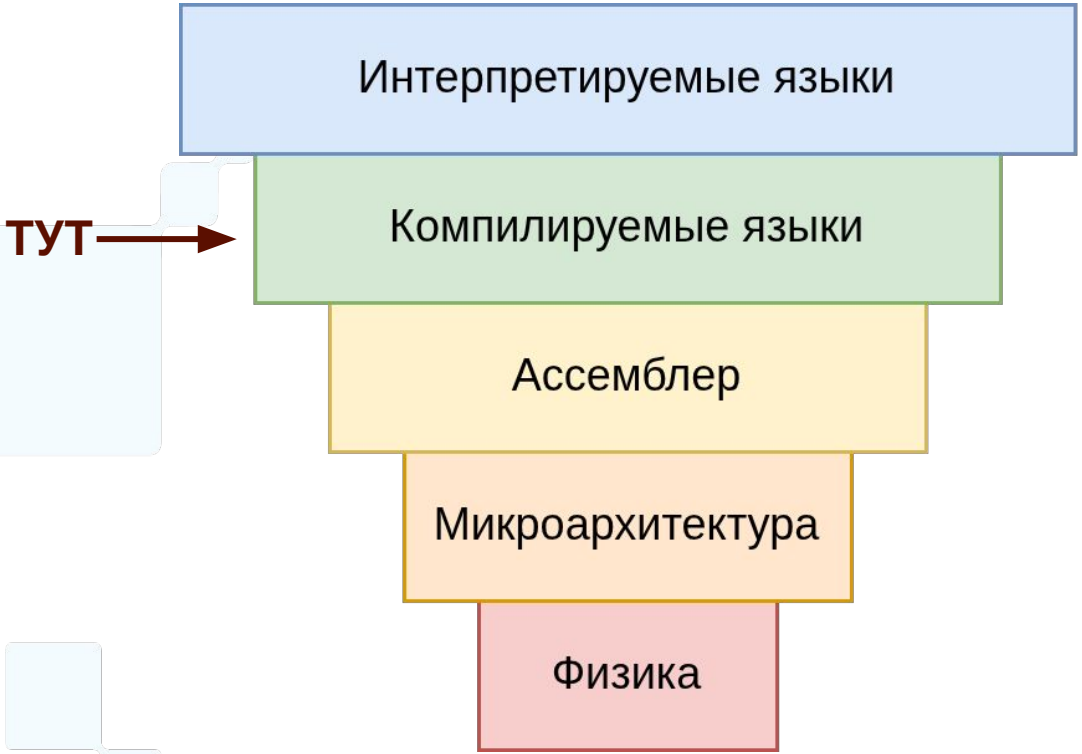
FAST FORWARD

АКОС

Лекция 4
Язык C

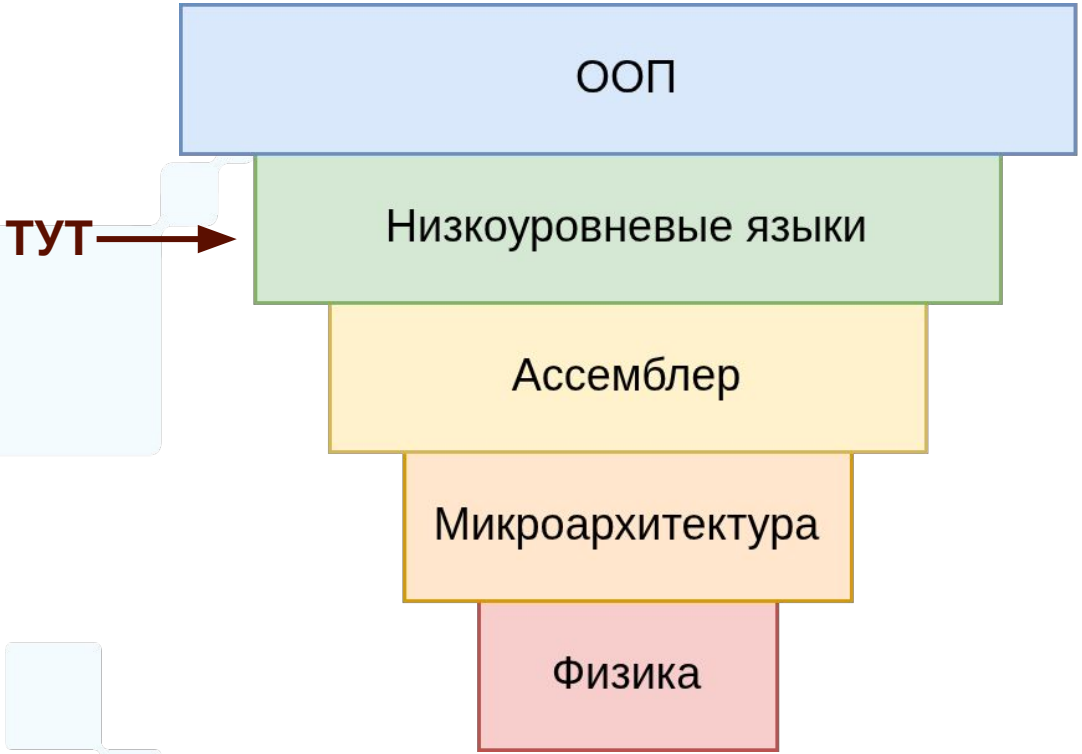
FAST FORWARD

Язык C



FAST FORWARD

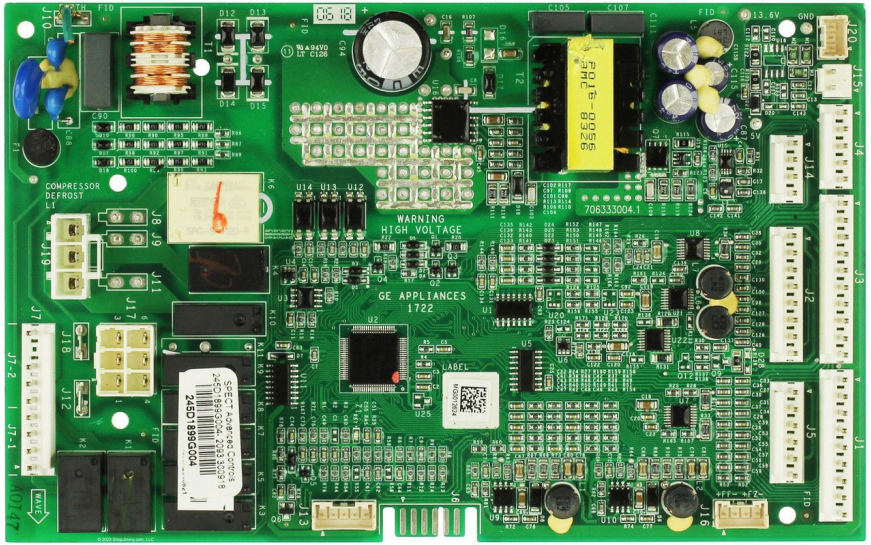
Язык С



FAST
FORWARD

Области применения

- операционные системы
- драйверы
- embedded / микроконтроллеры
- системные библиотеки
- компиляторы
- базы данных
- сетевое ПО
- high-performance code
- realtime системы



FAST
FORWARD

A C++?

VS

FAST FORWARD

Стандарт языка C

- Implementation-defined behavior

  представление целых чисел

  two's complement, sign-magnitude

- Unspecified behavior

  порядок вычисления аргументов при вызове функции

  f(g(), h())

- Undefined behavior (UB)

  выход за границы массива

FAST FORWARD

Целочисленные типы

char	signed char	unsigned char
short	signed short	unsigned short
int	signed int	unsigned int
long	signed char	unsigned char
long long	signed long long	unsigned long long

FAST FORWARD

Функции

```
int foo();                // Объявление без реализации

int bar() {               // С реализацией
    return 3;
};
```

FAST FORWARD

Переменные

```
int x;              // Глобальная

void foo() {
    int y;          // Локальная
}
```

FAST FORWARD

Структуры

```
struct example {
    char b;
    int i;
    long long l;
};

struct example p1 = {1, 2, 3};
struct example p2 = {.b = 1, .l = 2};
```

Массивы

```
int arr[3] = {1, 2, 3};

// кол-во элементов = кол-во инициализированных

int arr[] = {1, 2, 3};
```

FAST FORWARD

Указатели

```
int x;
int *p = &x;    // адрес переменной
*p = 10;        // разыменовать
```

FAST FORWARD

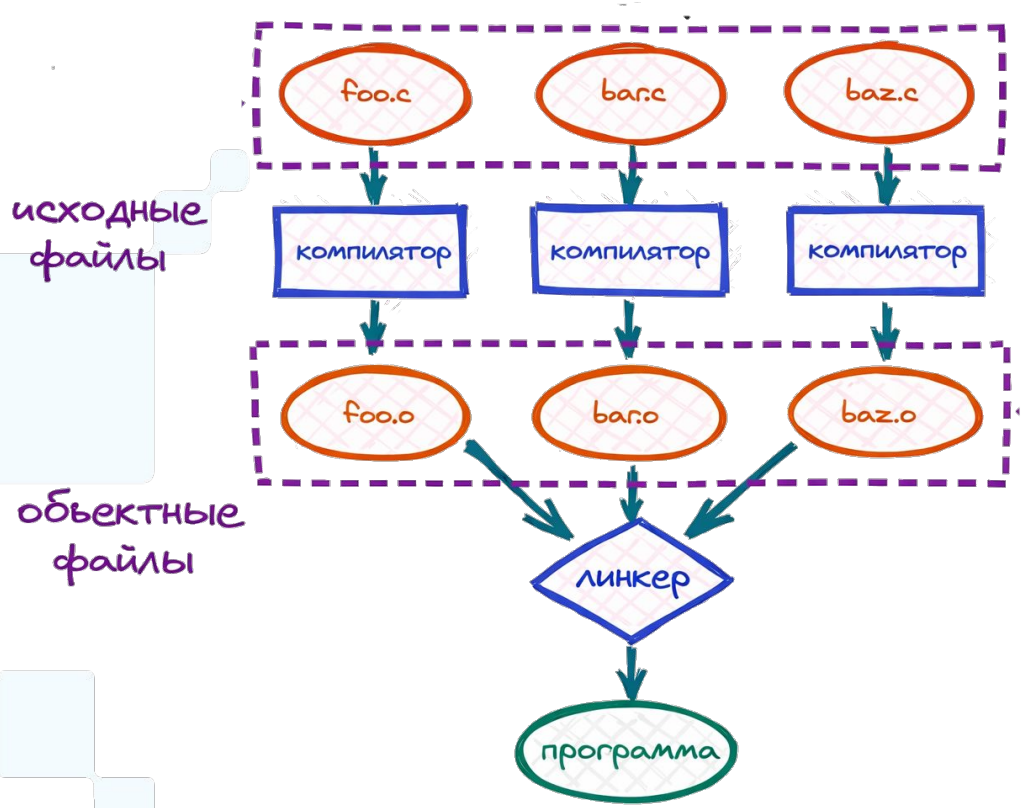
Пример

```
int main() {}
```

```
gcc hello.c -o hello  // компиляция
./hello               // запуск
```

FAST FORWARD

Сборка



FAST FORWARD

Целочисленные типы

char	**CHAR_BIT**
short	min **16** bit
int	min **16** bit
long	min **32** bit
long long	min **64** bit

FAST FORWARD

Целочисленные типы

Тип	Atmel AVR	32-bit	**Win64**	**64-bit**
char	1	1	**1**	**1**
short	2	2	**2**	**2**
int	2	4	**4**	**4**
long	4	4	**4**	**8**
long long	-	8	**8**	**8**
__int128	-	-	**-**	**16**

FAST
FORWARD

<stdint.h>

int8_t, int16_t, int32_t, int64_t

uint8_t, uint16_t, uint32_t, uint64_t

intptr_t, uintptr_t

int8_min_t

int8_fast_t

FAST FORWARD

Нюансы

char	signed char ? unsigned char
short	signed short
int	signed int
long	signed char
long long	signed long long

FAST FORWARD

<limits.h>

char	CHAR_MIN, CHAR_MAX, SCHAR_MIN, SCHAR_MAX, UCHAR_MIN
short	SHRT_MIN, SHRT_MAX, USHRT_MAX
int	INT_MIN, INT_MAX, UINT_MAX
long	LONG_MIN, LONG_MAX, ULONG_MAX
long long	LLONG_MIN, LLONG_MAX, ULLONG_MAX

FAST FORWARD

Константы

```
10          // int
2147483648  // на gcc/amd64 – тип long, на x86_32 – тип long long
0x80000000  // то же число, но тип константы unsigned int
```

Первый тип из списка

10ричные - int, long, long long

8/16ричные - int, uint, long, ulong, llong, ullong

FAST FORWARD

Константы

```
0      // int
0U     // unsigned
0L     // long
0UL    // unsigned long
0LU    // unsigned long
0LL    // long long
0ULL   // unsigned long long
0LLU   // unsigned long long
```

FAST FORWARD

Арифметика

Unsigned - по модулю 2 ^ N

Signed - неправильный результат - UB

```
UINT_MAX + 1;   // always 0
INT_MAX + 1;    // UB
i < i + 1;      // always true?
```

Арифметика. Опции

-ftrapv

abort() при знаковом целочисленном переполнении

-fwrapv

Выполнять знаковые операции по модулю 2^N, но компилятор не делает никаких оптимизационных предположений

-fsanitize=undefined

Частичная проверка на undefined behavior при работе программы

FAST FORWARD

Implicit conversions

```
unsigned char x = 255;
unsigned char y = 1;
int res = x + y;
```

~~0~~ 256

FAST FORWARD

Функции

```
int foo();          // объявление без реализации
int bar() {};       // С реализацией
int baz(int);       // Принимающее одно число

int empty(void);    // Не принимающее аргументов
int any();          // Любое количество аргументов
```

FAST FORWARD

Переменные

```
int x;                  // статический класс хранения
extern int ext;         // внешний класс хранения
                        // (объявление без определения)

void foo(void) {
    int y;              // автоматический
    static int z;       // статический
    register int t;     // регистровый
}
```

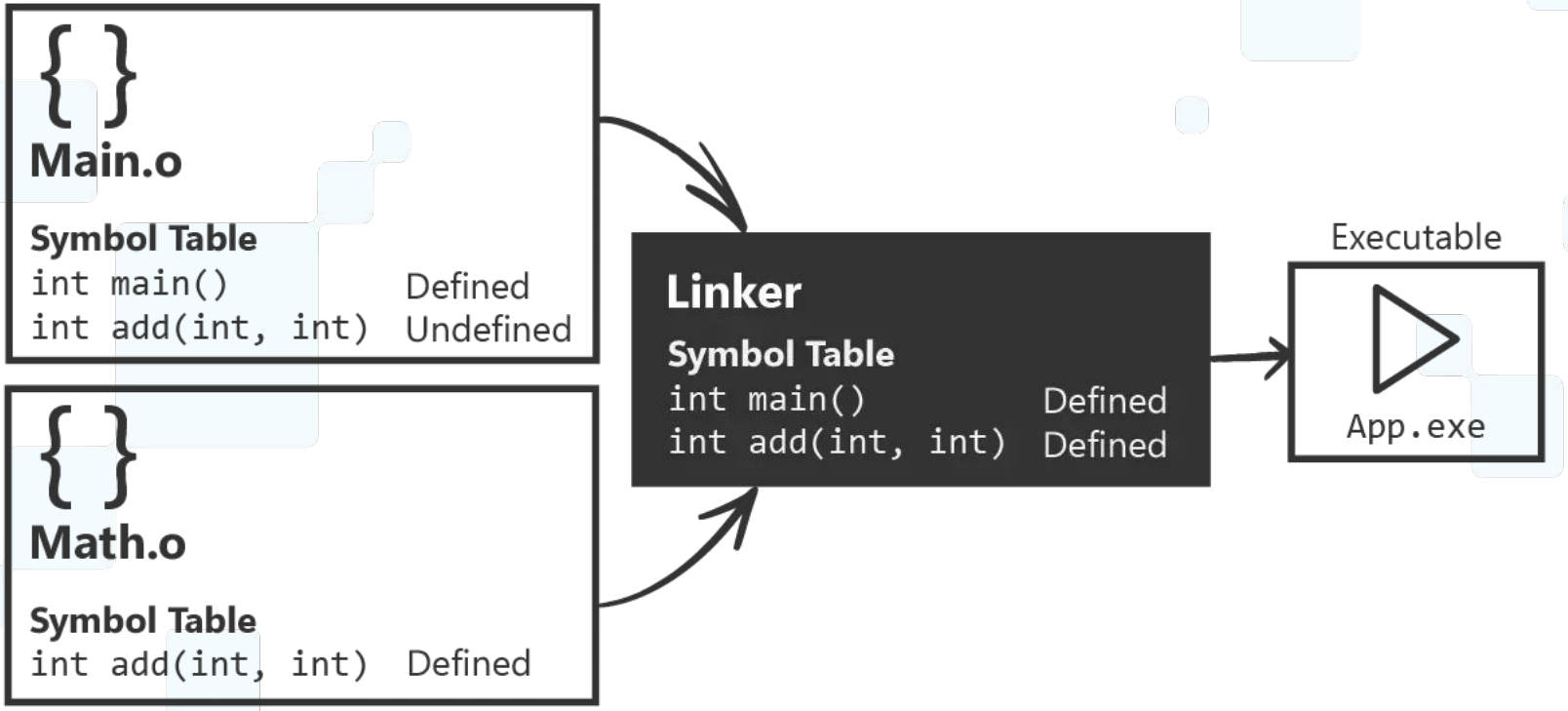
FAST FORWARD

Пример

```
// foo.c
extern int x;
int fun(int a, int b);


// bar.c
int x;
int fun(int a, int b) {}
```

FAST FORWARD

Линковка



FAST FORWARD

Пример

```
int foo() {}
int main() {}
```

```
gcc hello.c -o hello  // компиляция
./hello               // запуск
gcc -c hello.c        // объектный файл
gcc -S hello.c        // ассемблерный файл
gcc -E hello.c        // остановка на стадии препроцессирования
gcc -Wall -Werror     // show all warnings, warnings as errors
```

FAST FORWARD

Static

```
// Видны в других единицах трансляции
int var;
int foo(int arg) {}

// Не видны
static int var2;
static void bar(void) {}
```

FAST FORWARD

Препроцессор. include

```
#include <stdio.h>  // системный
#include "some.h"  // пользовательский

#pramga once        // Против рекурсии

#ifndef FOO_H       // Против рекурсии
#define FOO_H
...
#endif
```

Препроцессор. Макросы

```
#define SHORT_MAX 32767

int func_returning_SHORT_MAX() {
    return SHORT_MAX;
}
```

FAST FORWARD

Препроцессор. Макросы

```
#define MYVAL 100 + 500

int func() {
    return MYVAL * 10;
}
```

Препроцессор. Макросы

```
#define MYVAL (100 + 500)

int func() {
    return MYVAL * 10;
}
```

FAST FORWARD

Препроцессор. Функции

```
#define SUM(a, b) ((a) + (b))

SUM(12 + 13, 100500)


#define DUM(a, b, c) SUM(SUM(a, b), c)

DUM(1, 2, 3)
```

FAST FORWARD

Структуры

```
struct example {
    char b;
    int i;
    long long l;
};

struct example p1 = {1, 2, 3};
struct example p2 = {.b = 1, .l = 2};
```

Выравнивание

1) Всегда степень 2
2) По размеру типа
3) Не более размера указателя
4) Между - неиспользуемые пустоты
5) Размер структуры кратен выравниванию
6) Выравнивание структуры - max от выравнивания полей

```
alignof(char)        == 1
alignof(short)       == 2
alignof(int)         == 4
alignof(long)        == 8
alignof(long long)   == 8
```

FAST
FORWARD

Выравнивание

```
struct example {  // aligned to 8 bytes
    char b;
    // 3 padding bytes
    int i;
    // no padding bytes
    long long l;
    char c;
    // 7 padding bytes
};
sizeof(struct example) == 24;  alignof(struct example) == 8;
```

FAST FORWARD

Выравнивание

```
struct example {  // aligned to 4 bytes
    char b;
    // 3 padding bytes
    int i;
    // no padding bytes
    char c;
    // 3 padding bytes
};
sizeof(struct example) == 12;  alignof(struct example) == 4;
```

FAST FORWARD

НЕ выравнивание

```
struct example {  // aligned to 8 bytes
    char b;
    int i;
    long long l;
    char c;
    // 2 padding bytes
} __attribute__((packed));

sizeof(struct example) == 16;
alignof(struct example) == 8;
```

FAST FORWARD

Массивы

```
int arr[10] = {};   // пустой инициализатор невалиден
int arr[10] = {0}; // остальные автоматически нулями
int arr[10] = {[5] = 4}; // и тут


// кол-во элементов = кол-во инициалированных
int arr[] = {1, 2, 3};
```

FAST FORWARD

Размер массива…

```
int a[10] = {0};

sizeof(a)    == 40;
sizeof(a[0]) == 4;

#define ARRLEN(a) (sizeof(a) / sizeof((a)[0]))
```

FAST FORWARD

Указатели

```
int x;
int *p = &x;    // адрес переменной
*p = 10;        // разыменовать
```

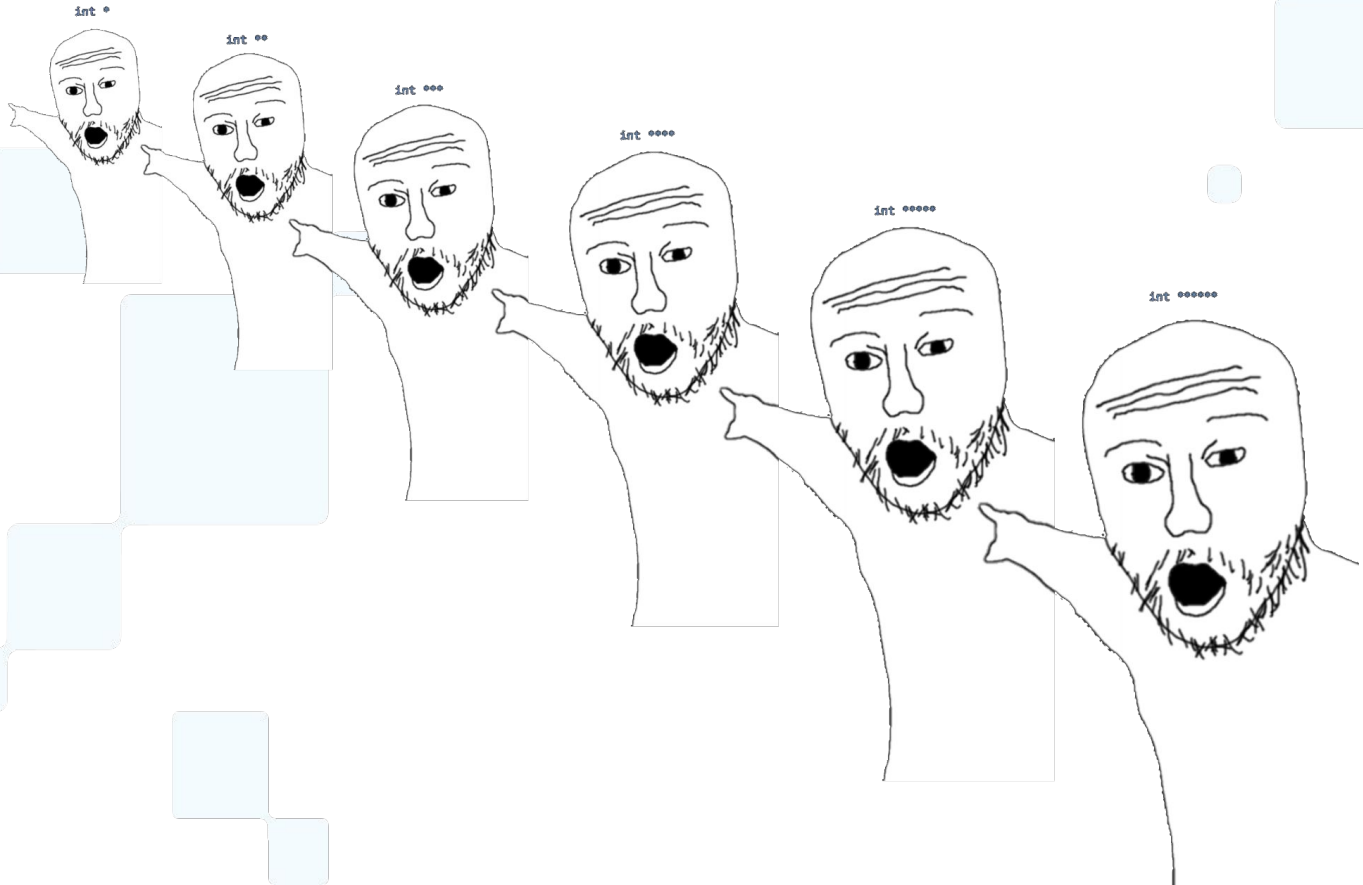
FAST FORWARD

Указательная арифметика

```
int* p;

*(p + 10) == p[10];

p + 10 == (intptr_t)p + 10 * sizeof(*p);


int *d;

size_t len = p - d;
```

FAST FORWARD

Указатели



FAST FORWARD

Указатели

```
int arr[10];

int* p = &arr[0];  // указатель

int** pp = &p;     // указатель на указатель
```

FAST FORWARD

Си-Строки

- Последовательность байт (char)
- Один байт - один “символ”
- Оканчивается нулём
- Размер не хранится
- Обычно подразумевают указатель на начало

FAST FORWARD

Си-Строки

```
char[] c = "hello world\n"; // на стеке
c[0] = 'H'; // можно менять, но длину - нельзя

// не получится поменять, лежит в rodata
char* str = "hello world\n";
```

~~str[0] = 'H';~~

FAST FORWARD

Длина

```
size_t strlen(char* s) {
    size_t i;
    for (i = 0; s[i] != '\0'; ++i);
    return i;
}
```

FAST FORWARD

printf

```
int printf(const char *restrict format, ...);
```

- Принимает строчку с форматом
- Читает строку, печатает обычные символы, пока не дойдет до символа %
- Смотрит спецификатор формата

FAST
FORWARD

printf

```
int printf(const char *restrict format, ...);
```

"%d" - int

"%s" - char*

"%x" - int в 16-ричной, "%o" - в 8-ричной

h - short

l - long int

FAST FORWARD

printf

```
int x = 10;
printf("%d\n", x);


short s = 7;
printf("%ld %hd\n", strlen(str), s);
// integer promotion of s
```

scanf

```
int scanf(const char *restrict format, ...);
```

- Очень важен спецификатор размера числа – в printf значения расширяются до размера int, а тут указатели.
- Возвращает кол-во считанных полей (если ни одного - 0).
- Особый случай: если ни одного поля не считано и достигнут **конец ввода**, возвращает специальное значение EOF (Ctrl-D).

FAST FORWARD

Ввод числа

```
short shint = 7;
int result = scanf("%hd", &shint);
if (result == EOF) {
    printf("empty input\n");
    return 1;
}
if (result != 1) {
    printf("error\n");
    return 1;
}
```

FAST FORWARD

Ввод строки

```
char buf[100];

int result = scanf("%s", buf);
if (result != 1) {
    printf("error\n");
    return 1;
}
printf("buf : %s\n", buf);
```

FAST FORWARD

Ввод строки

```
char buf[100];

scanf("%s", buf);

scanf("%99s", buf);
```

FAST FORWARD

АКОС

QCourse.ru 2026